

কীভাবে Transformers বিভিন্ন টাস্ক সমাধান করে (How Transformers solve tasks)

আগের সেকশনে “Transformers কী কী করতে পারে?”—এখানে আপনি NLP, স্পিচ/অডিও, কম্পিউটার ভিশন এবং সেগুলোর কিছু গুরুত্বপূর্ণ ব্যবহার সম্পর্কে জেনেছেন। এই পেজে আমরা আরও এক ধাপ ভেতরে যাব—মানে মডেলগুলো আসলে ভিতরে কীভাবে কাজ করে এবং কোন প্রক্রিয়ায় একটা টাস্কের জন্য ব্যবহারযোগ্য আউটপুট দেয়, সেটা বুঝব।

একটা টাস্ক সমাধানের অনেক রকম পদ্ধতি থাকতে পারে। কোনো কোনো মডেল নতুন টেকনিক যোগ করে বা ভিন্নভাবে সমস্যাটা অ্যাপ্রোচ করে। কিন্তু Transformer মডেলগুলোর ক্ষেত্রে সাধারণ ধারণাটা প্রায় একই—কারণ Transformer আর্কিটেকচার খুবই ফ্লেক্সিবল। তাই অধিকাংশ Transformer মডেলকে আপনি তিনটা কাঠামোর ভেতরে ফেলতে পারবেন:

- **Encoder**
- **Decoder**
- **Encoder–Decoder**

১) সব টাস্কের সাধারণ “প্যাটার্ন” কী?

Transformer দিয়ে যেসব টাস্কই করুন না কেন, বেশিরভাগ ক্ষেত্রে একই রকম একটা ক্রো দেখা যায়:

1. ইনপুট ডেটা প্রস্তুত করা (**data preparation / preprocessing**)
টেক্সট হলে tokenization, অডিও হলে feature extraction, ইমেজ হলে resizing/normalization—এ ধরনের কাজ।
2. মডেলের ভেতর দিয়ে ডেটা চালানো (**model forward pass**)
Encoder/Decoder/Encoder–Decoder—যেটা লাগবে, সেটার মাধ্যমে ইনপুট প্রসেস হয়।
3. আউটপুটকে টাস্ক অনুযায়ী ব্যাখ্যা করা (**postprocessing / interpretation**)
মডেল যে raw output দেয় (logits/probabilities/boxes/masks), সেটাকে মানুষের ব্যবহারযোগ্য ফলাফলে রূপান্তর করা হয়।

অর্থাৎ, মূল পার্থক্য তৈরি হয় এই তিন জায়গায়:

- ইনপুট কীভাবে প্রস্তুত করা হচ্ছে
- কোন আর্কিটেকচার ব্যবহার হচ্ছে (encoder / decoder / encoder-decoder)
- আউটপুট কীভাবে টাস্ক অনুযায়ী প্রসেস করা হচ্ছে

২) এখানে আমরা কোন কোন মডেল ও টাস্ক দেখব?

এই পেজে কয়েকটা জনপ্রিয় মডেল এবং তাদের টাস্কগুলো উদাহরণ হিসেবে ধরা হয়েছে:

স্পিচ/অডিও

- **Wav2Vec2:**
 - Audio classification
 - Automatic Speech Recognition (ASR)

কম্পিউটার ভিশন (ইমেজ)

- **Vision Transformer (ViT)** এবং **ConvNeXT**: Image classification
- **DETR**: Object detection
- **Mask2Former**: Image segmentation
- **GLPN**: Depth estimation

NLP / টেক্সট

- **BERT**: Encoder-based টাস্ক
 - Text classification
 - Token classification
 - Question answering
- **GPT-2**: Decoder-based টাস্ক
 - Text generation
- **BART**: Encoder–Decoder-based টাস্ক
 - Summarization
 - Translation

৩) আগে কী জানা থাকা ভালো?

এখানে ঢোকান আগে Transformer-এর বেসিক ধারণা জানা থাকলে বুঝতে সুবিধা হবে:

- Encoder কী, Decoder কী
- Attention কীভাবে কাজ করে
- Encoder-only / Decoder-only / Encoder–Decoder কার্ণামোর পার্থক্য

এই বেসিকগুলো জানা থাকলে পরে বিভিন্ন মডেল কেন একেক টাস্কে ভালো কাজ করে—সেটা স্বাভাবিকভাবেই মাথায় বসে যাবে।

ভাষার জন্য Transformer মডেল (Transformer models for language)

আধুনিক NLP-এর “হাট” হলো **Language Model**। এগুলোর লক্ষ্য:

- মানুষের ভাষা বোঝা
- মানুষের মতো ভাষা তৈরি করা
- শব্দ/টোকেনের মধ্যে সম্পর্ক এবং পরিসংখ্যানগত প্যাটার্ন শেখা

Transformer প্রথম বানানো হয়েছিল **machine translation** করার জন্য। কিন্তু এখন এটি প্রায় সব AI টাস্কের ডিফল্ট আর্কিটেকচার হয়ে গেছে। কারণ:

- কিছু টাস্ক **encoder** দিয়ে ভালো হয়
- কিছু টাস্ক **decoder** দিয়ে ভালো হয়
- কিছু টাস্কে দুটোই লাগে, তাই **encoder-decoder** লাগে

৪) Language model কীভাবে কাজ করে?

Language model ট্রেন করা হয় এই ধারণার উপর:

“কনটেক্সট দেখে পরের শব্দ (বা নির্দিষ্ট শব্দ) কী হতে পারে—তার সম্ভাবনা বের করা”

একবার মডেল ভাষার এই “ভিত্তি” শিখে ফেললে, সেটা অনেক টাস্কে কাজে লাগানো যায়। এটাকেই সাধারণভাবে foundation বলা যায়।

Transformer language model ট্রেন করার দুইটা সবচেয়ে সাধারণ পদ্ধতি আছে:

৫) Transformer ট্রেনিং-এর দুই প্রধান পদ্ধতি

ক) Masked Language Modeling (MLM)

এটা সাধারণত **encoder** টাইপ মডেলে (যেমন BERT) ব্যবহৃত হয়।

পদ্ধতি:

- ইনপুট বাক্যের কিছু token এলোমেলোভাবে mask করে দেওয়া হয়
- তারপর মডেলকে শেখানো হয় সেই আসল token কী ছিল তা প্রেডিক্ট করতে

এতে লাভ:

- মডেল একই সাথে শব্দের আগের এবং পরের context ব্যবহার করতে শেখে
- এটাকে বলা হয় **bidirectional context**

সহজ কথা:

- “বাক্যের মাঝখানে একটা শব্দ লুকালে, চারপাশ দেখে বের করতে পারা”

খ) Causal Language Modeling (CLM)

এটা সাধারণত **decoder** টাইপ মডেলে (যেমন GPT) ব্যবহৃত হয়।

পদ্ধতি:

- মডেলকে শেখানো হয় আগের সব token দেখে পরের **token** প্রেডিক্ট করতে

এখানে নিয়ম:

- মডেল ভবিষ্যৎ token দেখতে পারবে না
- শুধু বাম দিকের (previous) context ব্যবহার করবে

এটা কেন দরকার:

- টেক্সট জেনারেশন করার সময় তো আপনি ভবিষ্যৎ জানেন না
- তাই generation-এর সাথে এই ট্রেনিং পদ্ধতি খুব ম্যাচ করে

৬) Transformers লাইব্রেরিতে Language model-এর ধরন

Transformers লাইব্রেরিতে language model সাধারণত তিন ধরনের আর্কিটেকচারে পড়ে:

৬.১ Encoder-only models (যেমন BERT)

বৈশিষ্ট্য:

- দুদিকের context দেখে (bidirectional) টেক্সট বুঝতে পারে
- “বোঝার” কাজগুলোতে সেরা

কোন কাজগুলোতে বেশি ভালো:

- Text classification (sentiment/spam/topic)
- Named Entity Recognition (NER)
- Question Answering (বিশেষ করে extractive QA)
- Token classification

৬.২ Decoder-only models (যেমন GPT, Llama)

বৈশিষ্ট্য:

- বাম থেকে ডানে (left-to-right) প্রসেস করে

- পরের token প্রেডিক্ট করে
- “লেখার/জেনারেট করার” কাজে সেরা

কোন কাজগুলোতে বেশি ভালো:

- Text generation
- Prompt completion
- Essay লেখা
- Code generation

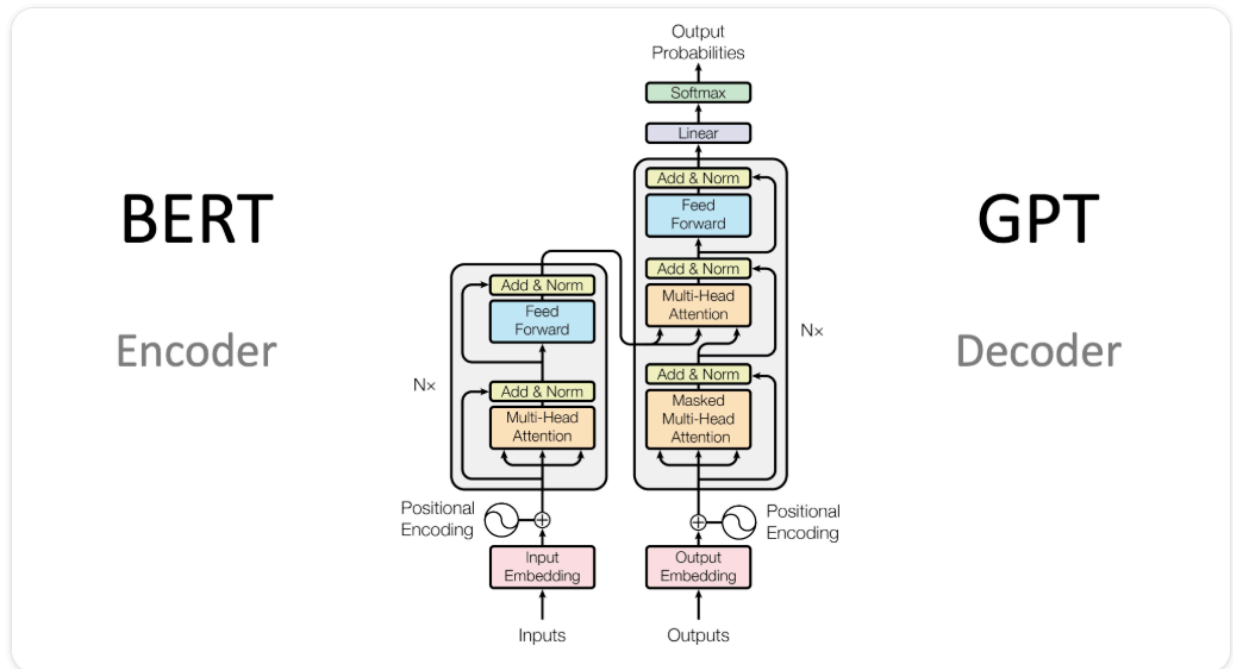
৬.৩ Encoder–Decoder models (যেমন T5, BART)

বৈশিষ্ট্য:

- Encoder ইনপুট “বোঝে”
- Decoder সেই বুঝে নেওয়া তথ্য ব্যবহার করে আউটপুট “তৈরি করে”
- Input → Output টাইপ টাস্কে সেরা (sequence-to-sequence)

কোন কাজগুলোতে বেশি ভালো:

- Translation
- Summarization
- Text-to-text টাস্ক
- (কিছু ক্ষেত্রে) Question Answering



৭) Pretraining → Fine-tuning: কেন এটা কাজ করে?

Transformer language model সাধারণত প্রথমে **pretrained** হয় বিশাল টেক্সট ডেটায়, **self-supervised** ভাবে (মানে মানুষের লেবেল লাগে না)।

তারপর নির্দিষ্ট টাস্কের জন্য **fine-tune** করা হয় ছোট task-specific ডেটায়।

এই প্রক্রিয়াটাকে বলা হয় **transfer learning**:

- pretrained মডেল ভাষার বেসিক “জ্ঞান” আগে থেকেই শিখে রাখে
- fine-tuning-এ সেই জ্ঞান ব্যবহার করে দ্রুত টাস্ক-এ অ্যাডাপ্ট করে

ফলে:

- কম ডেটা লাগে
- কম compute লাগে
- ভালো ফল পাওয়া যায়

৮) সামনে কী আসছে?

পরের অংশগুলোতে আমরা স্পেসিফিক আর্কিটেকচার এবং তাদের টাস্ক অ্যাপ্লিকেশন দেখব—

- Speech (ASR, audio classification)
- Vision (classification, detection, segmentation, depth)
- Text (encoder/decoder/encoder-decoder টাস্ক)

লক্ষ্য হলো আপনি বুঝবেন:

- কোন টাস্কে কোন আর্কিটেকচার কেন ব্যবহার হচ্ছে
- ইনপুট/আউটপুট কেমন
- “under the hood” কী হচ্ছে

Text Generation (টেক্সট জেনারেশন)

Text generation মানে হলো—

একটি দেওয়া ইনপুট বা **prompt** থেকে এমন টেক্সট তৈরি করা, যেটা:

- অর্থপূর্ণ (coherent)
- কনটেক্সটের সাথে মানানসই (contextually relevant)

- মানুষের লেখা টেক্সটের মতো মনে হয়

আজকের বেশিরভাগ চ্যাটবট, লেখালেখি টুল, কোড জেনারেটর—সবকিছুর মূল ভিত্তিই হলো text generation।

১) GPT-2 কী এবং কেন এটা গুরুত্বপূর্ণ?

GPT-2 হলো একটি **decoder-only Transformer model**।

এর মানে:

- এতে শুধু **decoder** অংশ থাকে
- এটি বাম দিক থেকে ডান দিকে (left-to-right) টেক্সট প্রসেস করে
- পরের শব্দ কী হবে—সেটা ভবিষ্যদ্বাণী করে

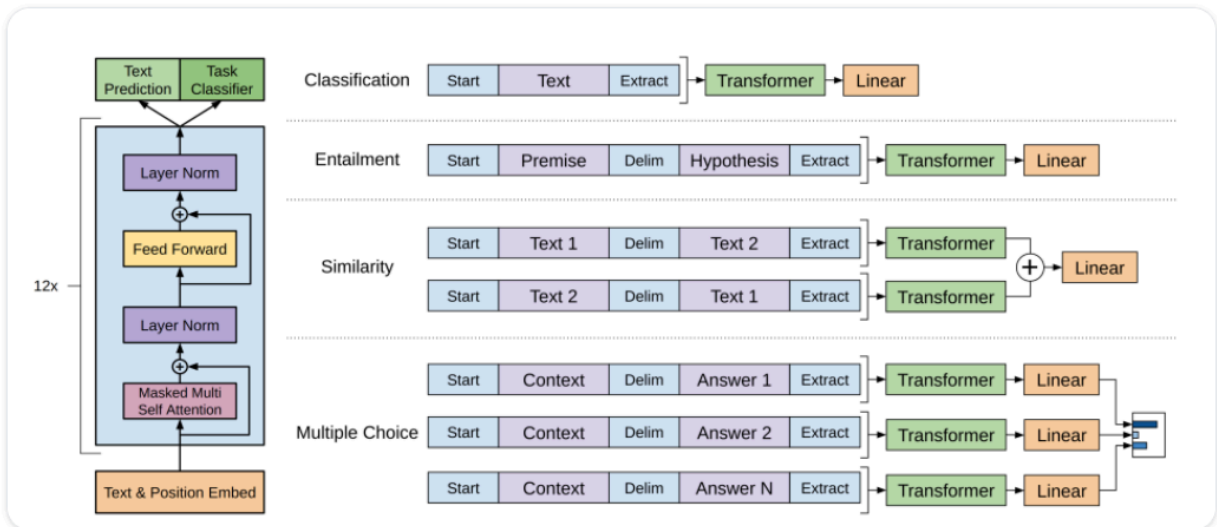
GPT-2 কে আগে থেকে বিশাল পরিমাণ টেক্সট ডেটায় **pretrain** করা হয়েছে।

এই কারণেই এটি:

- একটি prompt দিলে বিশ্বাসযোগ্য টেক্সট তৈরি করতে পারে
- এমনকি যেসব টাস্কে আলাদা করে ট্রেন করা হয়নি (যেমন question answering), সেগুলোও মোটামুটি ভালোভাবে করতে পারে

গুরুত্বপূর্ণ সতর্কতা:

- GPT-2 অনেক সময় খুব আত্মবিশ্বাসের সাথে ভুল তথ্যও লিখতে পারে
- কারণ এটি “সত্য বোঝে” না—শুধু ভাষার প্যাটার্ন শিখেছে



১) বাম পাশের অংশ: Transformer Encoder (মূল মডেল)

বাম দিকে নীল রঙের বড় ব্লকটি হলো **Transformer Encoder stack**। এটাকেই মডেলের মূল “বোঝার ক্ষমতা” বলা যায়।

(ক) Text & Position Embedding

- ইনপুট টেক্সট প্রথমে token embedding এ রূপান্তর করা হয়
- এরপর position embedding যোগ করা হয়
- কারণ Transformer নিজে থেকে শব্দের ক্রম বা অবস্থান বোঝে না

(খ) Masked Multi Self-Attention

- প্রতিটি শব্দ বাক্যের অন্যান্য শব্দের দিকে তাকায়
- কোন শব্দ কোনটার সাথে কতটা সম্পর্কিত তা শেখে
- Encoder হওয়ায় এখানে পুরো বাক্য একসাথে দেখা যায়

(গ) Layer Normalization এবং Feed Forward Network

- Attention থেকে পাওয়া তথ্যকে আরও স্থিতিশীল ও শক্তিশালী করা হয়
- Feed Forward Network প্রতিটি token-এর representation আরও গভীর করে
- Layer Normalization ট্রেনিং স্থিতিশীল রাখে

(ঘ) 12× লেখা থাকার মানে

- এই attention + feedforward ব্লকটি ১২ বার পুনরাবৃত্তি করা হয়েছে
- এটি সাধারণত BERT-base এর মতো মডেলের লেয়ার সংখ্যা বোঝায়

(ঙ) উপরের Task-specific head

- Transformer শুধু representation তৈরি করে
- আসল কাজ (prediction বা classification) করে উপরের আলাদা head
- যেমন Text Prediction বা Task Classifier

২) ডান পাশের অংশ: বিভিন্ন NLP টাস্ক

ডান দিকে দেখানো হয়েছে—একই Transformer encoder কীভাবে বিভিন্ন NLP টাস্কে ব্যবহার হয়।

টাস্ক ১: Text Classification

ইনপুট ফরম্যাট

[Start] Text [Extract]

ব্যাখ্যা

- পুরো টেক্সট Transformer দিয়ে প্রসেস হয়
- [Extract] (বা [CLS]) টোকেনের final representation নেওয়া হয়
- সেটাকে Linear layer এ পাঠানো হয়
- Linear layer থেকে পাওয়া যায় ক্লাস লেবেল

উদাহরণ:

- Positive / Negative
- Spam / Not Spam
- Topic classification

টাস্ক ২: Entailment (Natural Language Inference)

ইনপুট ফরম্যাট

[Start] Premise [Delim] Hypothesis [Extract]

ব্যাখ্যা

- দুটি বাক্য একসাথে Transformer এ দেওয়া হয়
- মডেল বোঝে তাদের সম্পর্ক কী
- আউটপুট হতে পারে:
 - Entails
 - Contradicts
 - Neutral

সবশেষে [Extract] টোকেন Linear layer এ গিয়ে ক্লাস নির্ধারণ করে।

টাস্ক ৩: Text Similarity

ইনপুট ফরম্যাট (দুইবার চালানো হয়)

[Start] Text1 [Delim] Text2 [Extract]

[Start] Text2 [Delim] Text1 [Extract]

ব্যাখ্যা

- Similarity দুই দিক থেকেই বোঝা দরকার
- তাই Text1–Text2 এবং Text2–Text1 দুবার প্রসেস করা হয়
- দুই আউটপুট যোগ করা হয়
- Linear layer similarity score বের করে

টাস্ক ৪: Multiple Choice Question Answering

ইনপুট ফরম্যাট (প্রতিটি অপশনের জন্য)

[Start] Context [Delim] Answer_i [Extract]

ব্যাখ্যা

- প্রতিটি উত্তর অপশন আলাদা করে Transformer এ পাঠানো হয়
- প্রতিটির জন্য একটি স্কোর তৈরি হয়
- সব স্কোর তুলনা করে সবচেয়ে ভালো উত্তর বেছে নেওয়া হয়

Start, Delim, Extract টোকেনগুলোর অর্থ

টোকেন

কাজ

Start সিকোয়েন্স শুরু নির্দেশ করে

Delim দুই অংশ আলাদা করে

Extract এখান থেকে final representation নেওয়া হয়

বাস্তবে এগুলো সাধারণত:

- Start $\approx$ [CLS]
- Delim $\approx$ [SEP]
- Extract $\approx$ [CLS] output

২) GPT-2 কীভাবে টেক্সটকে টোকেনে ভাঙে?

GPT-2 সরাসরি শব্দ নিয়ে কাজ করে না। প্রথমে টেক্সটকে ভাঙতে হয় ছোট ছোট ইউনিটে, যেগুলোকে বলা হয় **token**।

এখানে GPT-2 ব্যবহার করে:

Byte Pair Encoding (BPE)

BPE হলো এমন একটি tokenization পদ্ধতি যেখানে:

- শব্দকে ছোট ছোট অংশে ভাঙা হয়
- খুব কমুন অংশগুলো একসাথে রাখা হয়
- অজানা শব্দও ভেঙে ভেঙে বোঝা সম্ভব হয়

এরপর:

- প্রতিটি token কে একটি **token embedding** এ রূপান্তর করা হয় (মানে সংখ্যার ভেক্টর)

৩) Positional Encoding কেন দরকার?

Transformer নিজে থেকে বুঝতে পারে না:

- কোন শব্দ আগে
- কোন শব্দ পরে

তাই token embedding—এর সাথে যোগ করা হয় **positional encoding**।

এর কাজ:

- প্রতিটি token বাক্যের কোন অবস্থানে আছে, সেটা মডেলকে জানানো

Token embedding + positional encoding মিলিয়ে তৈরি হয়:

Input embedding

৪) Decoder blocks—এর ভেতরে কী হয়?

এই input embedding গুলো GPT-2—এর একাধিক **decoder block** এর ভেতর দিয়ে যায়।

প্রতিটি decoder block—এর সবচেয়ে গুরুত্বপূর্ণ অংশ হলো:

Masked Self-Attention

এখানে দুইটা বিষয় খুব গুরুত্বপূর্ণ:

ক) Self-attention

- একটি token অন্য token—গুলোর দিকে তাকিয়ে নিজের অর্থ ঠিক করে
- কনটেক্সট বুঝতে সাহায্য করে

খ) Masked কেন?

GPT-2 কখনোই **future token** দেখতে পারে না।

মানে:

- “আমি আজ ____ খাব”
- মডেল যখন “আজ” প্রসেস করছে, তখন “খাব” দেখতে পারবে না

এটা করা হয়:

- একটি **attention mask** ব্যবহার করে
- future token—গুলোর attention score শূন্য (0) করে দিয়ে

এটা BERT-এর [MASK] token থেকে আলাদা:

- BERT—এ শব্দ লুকানো থাকে
- GPT-2—এ শব্দ লুকানো না, বরং দৃষ্টি (**attention**) বন্ধ করে দেওয়া হয়

৫) Decoder থেকে Language Modeling Head

সব decoder block পেরিয়ে আমরা পাই:

- একটি **final hidden state**

এই hidden state সরাসরি শব্দ নয়।

তাই এটাকে পাঠানো হয়:

Language Modeling Head—এ

এখানে:

- একটি linear transformation করা হয়
- hidden state → **logits** (প্রতিটি token-এর সম্ভাবনা স্কোর)

৬) Next token কীভাবে শেখানো হয়?

GPT-2 শেখে এইভাবে:

- ইনপুট:
I love machine
- লেবেল (target):
learning

ট্রেনিংয়ের সময়:

- logits এক ধাপ ডানে shift করা হয়
- অর্থাৎ, প্রতিটি অবস্থানে “পরের token” কে লেবেল ধরা হয়

এরপর:

- predicted logits এবং আসল token-এর মধ্যে **cross-entropy loss** হিসাব করা হয়

এই loss কমাতে কমাতে মডেল শিখে যায়:

“এই কনটেক্সটে পরের token কী হওয়ার সম্ভাবনা বেশি”

৭) GPT-2 কেন Text Generation-এ এত ভালো?

কারণ GPT-2-এর পুরো pretraining objective একটাই:

Causal Language Modeling

মানে:

- আগের সব token দেখে
- পরের token প্রেডিক্ট করা

এই লক্ষ্যটা সরাসরি **text generation**-এর সাথে মিলে যায়।

ফলে GPT-2:

- বাক্য সম্পূর্ণ করতে পারে
- গল্প/অনুচ্ছেদ লিখতে পারে
- prompt থেকে লম্বা টেক্সট তৈরি করতে পারে

Text Classification (টেক্সট ক্লাসিফিকেশন)

Text classification মানে হলো—একটি টেক্সট ডকুমেন্ট/বাক্যকে আগে থেকে নির্ধারিত কিছু ক্যাটাগরির মধ্যে ফেলা। যেমন:

- Sentiment analysis (Positive/Negative)
- Topic classification (Sports/Politics/Business)
- Spam detection (Spam/Not spam)

এই কাজগুলোতে সাধারণত মডেলকে পুরো টেক্সটটা “বোঝার” দরকার হয়—তাই এখানে **encoder-only** টাইপ Transformer, বিশেষ করে **BERT**, খুব ভালো কাজ করে।

১) BERT কেন Text classification—এ ভালো?

BERT একটি **encoder-only** মডেল এবং এটি প্রথম দিকের মডেলগুলোর মধ্যে অন্যতম, যেটা সত্যিকার অর্থে **deep bidirectionality** ব্যবহার করে।

Deep bidirectionality মানে:

- কোনো শব্দ বুঝতে গিয়ে BERT একই সাথে
 - বাম দিকের শব্দ (আগের শব্দ)
 - ডান দিকের শব্দ (পরের শব্দ)দুটোকেই বিবেচনা করে।

এটা GPT-type decoder-only মডেলের চেয়ে ভিন্ন, কারণ GPT সাধারণত শুধু বাম দিক দেখে (left-to-right)।

২) BERT কীভাবে ইনপুট টেক্সট প্রস্তুত করে?

BERT টেক্সটকে token-এ ভাঙতে ব্যবহার করে:

WordPiece Tokenization

WordPiece এমন একটি পদ্ধতি যেখানে:

- শব্দকে প্রয়োজনে ছোট ছোট সাব-ওয়ার্ডে (subword) ভেঙে নেওয়া হয়
- ফলে অজানা শব্দও ভেঙে বুঝতে পারে

এরপর প্রতিটি token থেকে তৈরি হয়:

- **token embedding**

বিশেষ টোকেন: **[CLS]** এবং **[SEP]**

[CLS] টোকেন

- প্রতিটি ইনপুট সিকোয়েন্সের একদম শুরুতে বসে
- classification টাস্কে এই [CLS] টোকেনটাই সবচেয়ে গুরুত্বপূর্ণ

কারণ:

Encoder দিয়ে পুরো সিকোয়েন্স প্রসেস হওয়ার পর [CLS] টোকেনের final representation-এ পুরো বাক্যের “সারমর্ম/সংক্ষিপ্ত ধারণা” জমা হয় বলে ধরা হয়।

এই [CLS] token-এর final output-ই পরে classification head-এ যায়।

[SEP] টোকেন

BERT অনেক সময় “দুটি বাক্য একসাথে” ইনপুট হিসেবে নেয় (sentence pair tasks)। সেক্ষেত্রে sentence A আর sentence B আলাদা করতে **[SEP]** টোকেন ব্যবহার হয়।

অর্থাৎ:

- Sentence A ... [SEP] Sentence B ... [SEP]

Segment Embedding (Sentence A / Sentence B বোঝাতে)

যখন দুইটা বাক্য থাকে, তখন BERT প্রতিটি টোকেনকে বলে দেয়—

- এটা প্রথম বাক্যের টোকেন (Segment A)
- নাকি দ্বিতীয় বাক্যের টোকেন (Segment B)

এটা হয় **segment embedding** দিয়ে।

৩) BERT কীভাবে Pretrain করা হয়?

BERT সাধারণত দুইটা objective দিয়ে pretrain করা হয়েছে:

Objective ১: Masked Language Modeling (MLM)

এখানে:

- ইনপুট টোকেনের কিছু অংশ র‍্যান্ডমভাবে mask করে দেওয়া হয়
- তারপর মডেলকে শেখানো হয়—mask করা আসল টোকেন কী ছিল

এটা কেন দরকার?

Bidirectional মডেলে একটা সমস্যা হয়:

- মডেল যদি সব শব্দ দেখতে পায়, তাহলে “প্রেডিক্ট” করা খুব সহজ হয়ে যায় (cheating)

MLM এই সমস্যা সামলায়:

- মডেলকে সত্যিকারভাবে context থেকে অনুপস্থিত শব্দ অনুমান করতে বাধ্য করে

মডেলের আউটপুটে:

- যেসব token mask করা হয়েছিল, তাদের final hidden state
- তারপর একটি feedforward network + softmax (vocab-এর উপর)
- ফলে কোন শব্দটা হওয়া উচিত—সেটা predict করা হয়

Objective ২: Next-Sentence Prediction (NSP)

এখানে মডেলকে দুইটা বাক্য দেওয়া হয়—A এবং B।

মডেলকে বলতে হয়:

- Sentence B কি সত্যিই Sentence A-এর পরে আসে? (IsNext)
- নাকি এটা random অন্য কোনো বাক্য? (NotNext)

ট্রেনিং সেটআপ:

- ৫০% সময় B সত্যিই পরের বাক্য
- ৫০% সময় B এলোমেলো বাক্য

আউটপুট:

- একটি feedforward network + softmax (দুই ক্লাস: IsNext / NotNext)

8) Encoder layers দিয়ে কী হয়?

Input embeddings (token + position + segment) একাধিক encoder layer দিয়ে যায়। শেষে আমরা পাই:

- প্রতিটি token-এর জন্য final hidden state

এর মধ্যে:

- [CLS] token-এর final hidden state হলো পুরো সিকোয়েন্সের প্রতিনিধিত্বকারী summary-type representation

৫) Text classification করতে হলে কী যোগ করতে হয়?

Pretrained BERT কে টেক্সট ক্লাসিফিকেশনের জন্য ব্যবহার করতে হলে, বেস BERT-এর উপরে বসানো হয়:

Sequence Classification Head

এটা সাধারণত একটি **linear layer**।

কাজ:

- [CLS] token-এর final hidden state নেয়
- linear transformation করে **logits** বানায় (প্রতিটি label-এর স্কোর)

এরপর:

- logits এবং আসল label-এর মধ্যে **cross-entropy loss** হিসাব হয়
- যার মাধ্যমে মডেল শিখে যায় কোন label সবচেয়ে সম্ভাব্য

Token Classification (টোকেন ক্লাসিফিকেশন)

Token classification মানে হলো—একটি সিকোয়েন্সের প্রতিটি **token**-এর জন্য আলাদা **label** দেওয়া।

উদাহরণ:

- Named Entity Recognition (PER/ORG/LOC)
- Part-of-speech tagging (NOUN/VERB/ADJ)

১) BERT দিয়ে Token classification কীভাবে হয়?

BERT চালিয়ে আমরা প্রতিটি token-এর final hidden state পাই।

তারপর বেস BERT-এর উপরে বসানো হয়:

Token Classification Head

এটাও সাধারণত একটি **linear layer**।

কাজ:

- প্রতিটি token-এর hidden state নেয়
- linear transformation করে logits দেয়
- প্রতিটি token-এর logits এর সাথে সঠিক token-label এর **cross-entropy loss** হিসাব হয়

ফলে:

- মডেল প্রতিটি token-এর জন্য সবচেয়ে সম্ভাব্য label বের করে

Question Answering (প্রশ্নোত্তর)

Question answering মানে হলো:

- একটি প্রশ্ন + একটি কনটেক্সট/প্যাসেজ দেওয়া থাকবে
- মডেল সেই কনটেক্সট থেকে প্রশ্নের উত্তর কোথায় আছে সেটি খুঁজে বের করবে

এটা সাধারণত “extractive QA”—মানে:

- মডেল নতুন করে উত্তর বানায় না
- কনটেক্সটের ভেতর থেকেই “answer span” কেটে বের করে

১) BERT দিয়ে QA কীভাবে হয়?

BERT ইনপুট হিসেবে সাধারণত নেয়:

- প্রশ্ন + কনটেক্সট (একসাথে, [SEP] দিয়ে আলাদা)

এরপর BERT থেকে প্রতিটি token-এর final hidden states পাওয়া যায়।

QA করতে বেস BERT-এর উপরে বসানো হয়:

Span Classification Head

এখানে linear layer দুটো গুরুত্বপূর্ণ আউটপুট দেয়:

- **start logits**: উত্তর কোন token থেকে শুরু
- **end logits**: উত্তর কোন token-এ শেষ

তারপর:

- predicted start/end position এর সাথে true start/end position এর **cross-entropy loss** হিসাব হয়
- সবচেয়ে সম্ভাব্য span নির্বাচন করা হয়

ফলাফল:

- কনটেন্টের ভিতর থেকে উত্তর অংশটা কেটে পাওয়া যায়

Summarization (সারসংক্ষেপ তৈরি)

Summarization মানে হলো—একটি বড় লেখা/আর্টিকেলকে ছোট করে লেখা, কিন্তু যেন:

- গুরুত্বপূর্ণ তথ্য (key information) বজায় থাকে
- মূল অর্থ (meaning) না বদলায়
- অপ্রয়োজনীয় অংশ বাদ পড়ে, কিন্তু “মেসেজ” ঠিক থাকে

এটা নিউজ সারাংশ, রিপোর্ট সারাংশ, ব্লগ সংক্ষেপ, ডকুমেন্ট রিভিউ—এ ধরনের কাজে খুব দরকারি।

১) Summarization কেন Encoder–Decoder মডেল দিয়ে ভালো হয়?

Summarization সাধারণত **sequence-to-sequence (seq2seq)** টাস্ক।

মানে:

- ইনপুট: লম্বা টেক্সট
- আউটপুট: ছোট সারাংশ টেক্সট

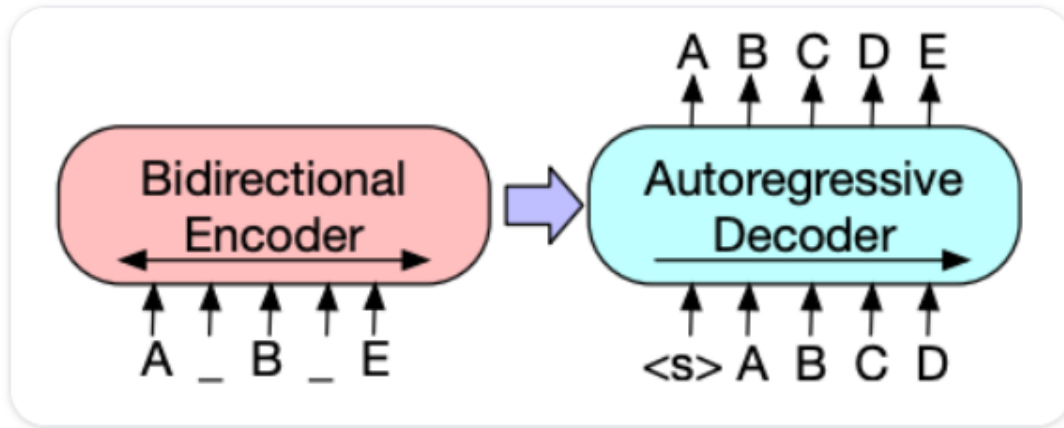
এখানে দুইটা কাজ একসাথে লাগে:

1. ইনপুট লেখাটা ভালোভাবে বোঝা (understanding)
2. সেই বুঝে নেওয়া তথ্য দিয়ে নতুন করে লেখা তৈরি করা (generation)

এই দুইটা কাজের জন্যই **encoder–decoder** আর্কিটেকচার আদর্শ।

তাই **BART** বা **T5**-এর মতো মডেল summarization-এ খুব শক্তিশালী।

এই অংশে আমরা BART কীভাবে কাজ করে সেটা বুঝব, পরে চাইলে T5 fine-tune করে হাতে-কলমে ট্রাই করা যায়।



২) BART-এর Encoder অংশ কীভাবে কাজ করে?

BART-এর encoder আর্কিটেকচার অনেকটাই BERT-এর মতো।

এর মানে:

- ইনপুট টেক্সট tokenization হয়ে token embedding তৈরি হয়
- সাথে positional embedding যোগ হয়
- তারপর encoder layers দিয়ে প্রসেস হয়
- ফলে ইনপুটের final hidden states (representation) পাওয়া যায়

তবে BART-এর pretraining কৌশল BERT থেকে আলাদা—এটাই বড় পার্থক্য।

৩) BART কীভাবে Pretrain করা হয়?

BART কে pretrain করা হয় “denoising” আইডিয়ায়। সহজ ভাষায়:

আগে ইনপুট টেক্সটকে ইচ্ছাকৃতভাবে নষ্ট/বিকৃত (corrupt) করা হয়
তারপর decoder-কে শেখানো হয় সেই টেক্সটকে আবার আগের মতো ঠিক করে বানাতে (reconstruct)

এখানে **corruption** বলতে বোঝায়—টেক্সটের কিছু অংশ পরিবর্তন/মুছে/লুকিয়ে দেওয়া।

BART-এর সুবিধা হলো: এটা বিভিন্ন ধরনের corruption ব্যবহার করতে পারে।

Text Infilling — সবচেয়ে ভালো corruption strategy

প্র্যাকটিসে BART-এর জন্য “text infilling” খুব ভালো কাজ করে।

Text infilling মানে:

- টেক্সটের কয়েকটা অংশ (span) কেটে/মুছে দেওয়া হয়
- কিন্তু প্রতিটি কাটা span-এর জায়গায় একটা মাত্র **[mask] token** বসানো হয়

উদাহরণ (আইডিয়া বোঝাতে):

- Original: “This course will teach you about transformer models.”
- Corrupt: “This course will [mask] transformer models.”

এখানে গুরুত্বপূর্ণ ব্যাপার:

- মডেল শুধু কোন শব্দ বসবে সেটা না
- কতগুলো শব্দ বসবে (span length) সেটাও শিখে ফেলে
কারণ [mask] একটাই, কিন্তু ভেতরে missing token সংখ্যা একাধিক হতে পারে।

৪) BART-এর Encoder আউটপুট কেন BERT থেকে আলাদা?

BERT pretraining-এ (MLM) শেষে সাধারণত masked token প্রেডিক্ট করার জন্য vocab-এর উপর softmax সহ head থাকে।

কিন্তু এই অংশে উল্লেখ করা পয়েন্টটা হলো:

- BART encoder শুধু representation তৈরি করে
- শব্দ প্রেডিক্ট করার কাজটা মূলত decoder side-এ ঘটে
- তাই encoder-এর শেষে BERT-এর মতো করে “শেষে বসানো নির্দিষ্ট word prediction head” এখানে জোরালোভাবে থাকে না (মূল reconstruction টাস্ক decoder সম্পন্ন করে)

৫) Decoder কীভাবে original text restore করে?

Encoder যে hidden states তৈরি করে, সেটা decoder-এর কাছে যায়।

Decoder-এর কাজ:

- encoder-এর representation দেখে
- যে span গুলো mask করা হয়েছিল সেগুলোসহ পুরো original টেক্সট আবার তৈরি করা

এখানে decoder encoder-এর আউটপুট থেকে অতিরিক্ত context পায়, ফলে restoration সহজ হয়।

তারপর decoder-এর আউটপুট যায়:

Language Modeling Head

এটা:

- hidden states $\rightarrow$ linear transformation

- তারপর **logits** তৈরি করে (vocab-এর উপর)

Training-এ label কী?

- label হলো “পরের টোকেন”
- তাই token sequence সাধারণত ডানে এক ধাপ **shift** করে label বানানো হয়

এরপর:

- logits বনাম label → **cross-entropy loss**
- এই loss কমাতে কমাতে মডেল শেখে কীভাবে টেক্সট reconstruct করতে হয়

৬) Summarization করতে গেলে কী হয়?

Fine-tuning সময়ে:

- ইনপুট: বড় লেখা (article)
- আউটপুট: সারাংশ (summary)

Encoder বড় লেখার representation বানায়

Decoder সেই representation ধরে সারাংশ জেনারেট করে

এই কারণেই BART/T5 summarization-এর জন্য খুব natural fit।

Translation (অনুবাদ)

Translation মানে হলো—এক ভাষা থেকে আরেক ভাষায় লেখা রূপান্তর করা, কিন্তু:

- মূল অর্থ ঠিক রাখা
- প্রসঙ্গ/context ঠিক রাখা
- ভাষার ব্যাকরণ/স্টাইল অনুযায়ী সঠিক আউটপুট দেওয়া

এটাও sequence-to-sequence টাস্ক:

- Input sequence (source language)
- Output sequence (target language)

তাই translation-এর জন্যও **encoder-decoder** মডেল (BART, T5) খুব উপযুক্ত।

১) BART কীভাবে Translation-এর সাথে মানিয়ে নেয়?

এই অংশে বলা হয়েছে—BART translation-এ adapt করার সময় একটি অতিরিক্ত ধারণা ব্যবহার করে:

আলাদা একটি **Source Encoder** যোগ করা

- একটি নতুন **encoder** তৈরি করা হয়, যেটা source language (যেমন French) কে এমন embedding/representation এ ম্যাপ করবে
- যেটা পরে pretrained encoder–decoder pipeline দিয়ে target language এ decode করা সম্ভব

এই নতুন encoder:

- শুরুতে **randomly initialized** থাকে (শূন্য থেকে শুরু)

তারপর কী হয়?

Step 1: প্রথম ধাপে কেবল নতুন অংশগুলো ট্রেন করা

এই ধাপে:

- Source encoder
- positional embeddings
- input embeddings

এগুলোকে ট্রেন করা হয় **cross-entropy loss** দিয়ে, যা আসে final model output থেকে।

এ সময় উল্লেখ করা হয়েছে:

- মূল pretrained model parameters প্রথম ধাপে **frozen** রাখা হয় (মানে, pretrained অংশগুলোর weights পরিবর্তন করা হয় না)

Step 2: দ্বিতীয় ধাপে সবকিছু একসাথে ট্রেন করা

এরপর দ্বিতীয় ধাপে:

- পুরো model-এর parameters একসাথে train করা হয় যাতে সিস্টেমটা source→target translation-এ ভালোভাবে end-to-end শিখে যায়

২) mBART কী?

BART-এর পরে এসেছে **mBART**—একটা multilingual ভার্সন, যা:

- অনেক ভাষায় pretrained
- বিশেষভাবে translation-এর জন্য বেশি উপযোগী

Modalities (Modalities beyond text)

Transformers শুধু টেক্সটের জন্য নয়। Transformer আর্কিটেকচার এতটাই ফ্লেক্সিবল যে এটাকে অন্যান্য ডেটা **modality** তেও ব্যবহার করা যায়, যেমন:

- Speech এবং Audio
- Images
- Video

এই কোর্সে মূল ফোকাস থাকবে টেক্সটের উপর, তবে অন্য modality-গুলো সম্পর্কে ধারণা থাকলে আপনি বুঝতে পারবেন Transformer কেন এত শক্তিশালী এবং কেন আজকে প্রায় সব AI টাস্কে এটি ব্যবহৃত হচ্ছে।

Speech এবং Audio

চলুন শুরু করি **speech/audio** দিয়ে। টেক্সট বা ছবির তুলনায় speech/audio ডেটা হ্যান্ডেল করা কিছুটা আলাদা চ্যালেঞ্জ তৈরি করে, কারণ:

- অডিও মূলত সময়ের সাথে পরিবর্তিত হয় (time-series data)
- শব্দের উচ্চারণ, গতি, একসেন্ট, ব্যাকগ্রাউন্ড নয়েজ—সব মিলিয়ে বৈচিত্র্য অনেক বেশি
- টেক্সটের মতো “স্পষ্ট টোকেন সীমা” শুরুতেই থাকে না

এই modality-তে Transformer কীভাবে কাজ করে—এটা বোঝার জন্য সবচেয়ে জনপ্রিয় উদাহরণ হলো **Whisper**।

Whisper কী?

Whisper হলো একটি **encoder-decoder (sequence-to-sequence) Transformer model**, যেটা ৬৮০,০০০ ঘণ্টা লেবেলড অডিও ডেটা দিয়ে pretrained।

এই বিশাল pretraining ডেটার কারণে Whisper পারে:

- English সহ বহু ভাষায়
- অনেক অডিও টাস্কে
- **zero-shot** ভাবে ভালো পারফর্ম করতে

এখানে “zero-shot” মানে:

- আপনি আলাদা করে আপনার ডেটায় fine-tune না করেও
- সরাসরি মডেল চালিয়ে ভালো ফল পেতে পারেন

এই কারণেই মানুষ বলে:

Whisper out of the box কাজ করে

Whisper কেন “fine-tune ছাড়াই” কাজ করতে পারে?

কারণ Whisper-এর decoder অংশ encoder থেকে শেখা speech representation কে:

- টেক্সটে রূপান্তর (transcription)
- এক ভাষা থেকে অন্য ভাষায় অনুবাদ (speech translation)
- ভাষা শনাক্ত করা (language identification)

এ ধরনের কাজের জন্য ব্যবহারযোগ্য আউটপুটে “ম্যাপ” করতে পারে—আর সেটা খুব বড় ও বৈচিত্র্যময় ডেটায় pretraining হওয়ার ফল।

Whisper-এর দুইটি প্রধান অংশ (Components)

Whisper মডেলে মূলত দুইটা বড় ব্লক আছে:

১) Encoder: অডিও থেকে representation বানায়

Encoder-এর কাজ হলো input অডিওকে প্রসেস করে এমন একটি representation বানানো, যেটা মডেল বুঝতে পারে।

Whisper এখানে অডিওকে সরাসরি ওয়েভফর্ম হিসেবে encoder-এ দেয় না। আগে একটা গুরুত্বপূর্ণ রূপান্তর করে:

Raw audio → Log-Mel Spectrogram

- Raw audio waveform কে **log-Mel spectrogram** এ রূপান্তর করা হয়
- এটা এক ধরনের ফিচার রিপ্রেজেন্টেশন, যেখানে অডিওর ফ্রিকোয়েন্সি তথ্য সময়ের সাথে কেমন বদলাচ্ছে তা দেখা যায়

এরপর:

- এই spectrogram কে Transformer **encoder network** এর মধ্যে পাঠানো হয়
- encoder শেষে আমরা পাই একটি encoded speech representation (hidden states)

সহজ কথা:

Encoder “অডিওটা কী বলছে” তার একটি সমৃদ্ধ ফিচার ম্যাপ বানায়

২) Decoder: speech representation থেকে টেক্সট তৈরি করে

Decoder-এর কাজ হলো encoder-এর আউটপুট দেখে ধাপে ধাপে টেক্সট টোকেন জেনারেট করা।

এটা করে **autoregressive** স্টাইলে, অর্থাৎ:

- আগের যে টোকেনগুলো তৈরি হয়েছে সেগুলো দেখে
- পরের টোকেনটি কী হবে সেটা প্রেডিক্ট করে
- এভাবে একটার পর একটা টোকেন যোগ করে সম্পূর্ণ বাক্য তৈরি করে

Decoder আসলে একটি স্ট্যান্ডার্ড Transformer decoder:

- input: previous text tokens + encoder output
- output: next text token prediction

বিশেষ টোকেন কেন দরকার?

Whisper-এর decoder ইনপুটের শুরুতে কিছু **special tokens** দেওয়া হয়, যাতে মডেল বুঝতে পারে আপনি কোন কাজ চান।

যেমন:

- transcription (শুধু কথাকে টেক্সট বানানো)
- translation (কথাকে অন্য ভাষায় অনুবাদ করে লেখা)
- language identification (কোন ভাষায় কথা বলা হয়েছে)

এগুলোকে “steering tokens” ভাবে পারেন—মানে মডেলকে নির্দেশনা দেওয়া।

Whisper-এর শক্তির মূল কারণ: বিশাল pretraining ডেটা

Whisper pretrained হয়েছিল ওয়েব থেকে সংগ্রহ করা বড় ও বৈচিত্র্যময় লেবেলড অডিও ডেটাসেটে—মোট ৬৮০,০০০ ঘণ্টা।

এটা ছিল “large-scale, weakly supervised” ধরনের pretraining—যার মূল লাভ হলো:

- বিভিন্ন ভাষা
 - বিভিন্ন উচ্চারণ/অ্যাকসেন্ট
 - বিভিন্ন পরিবেশগত নয়েজ
 - বিভিন্ন স্পিচ স্টাইল
- এসব সামলে নেওয়ার সক্ষমতা তৈরি হয়

ফলে Whisper অনেক ক্ষেত্রে fine-tune ছাড়াই ভালো কাজ করে।

এখন **Whisper** আপনি কীভাবে ব্যবহার করতে পারেন?

Whisper pretrained হয়ে যাওয়ার পর আপনার কাছে দুইটা পথ থাকে:

1. **Zero-shot inference**

- সরাসরি মডেল চালিয়ে transcription/translation করা

2. **Fine-tuning**

- আপনার নিজের ডেটায় ট্রেন করে
- নির্দিষ্ট ডোমেইনে (যেমন call center, মেডিক্যাল স্পিচ, লোকাল অ্যাকসেন্ট) পারফরম্যান্স আরও বাড়ানো
- বিশেষভাবে Automatic Speech Recognition (ASR) বা Speech Translation টাস্কে

Automatic Speech Recognition (ASR)

Automatic Speech Recognition মানে হলো অডিও/স্পিচ ইনপুট থেকে স্বয়ংক্রিয়ভাবে লেখা (transcript) বের করা। প্রি-ট্রেন্ড (pretrained) মডেল দিয়ে ASR করতে গেলে আমরা সাধারণত মডেলের পুরো **encoder-decoder** কার্টামো ব্যবহার করি।

১) Whisper/Encoder-Decoder দিয়ে ASR কীভাবে কাজ করে?

এখানে মূল ধারণাটা খুব সোজা:

- **Encoder** অডিও ইনপুট প্রসেস করে (অডিও থেকে ফিচার/representation বানায়)
- **Decoder** সেই representation দেখে ধাপে ধাপে (autoregressive) টেক্সট তৈরি করে অর্থাৎ token by token করে পুরো transcript তৈরি হয়

Fine-tuning করলে কীভাবে ট্রেন হয়?

Fine-tuning-এর সময় মডেলকে সাধারণত **sequence-to-sequence loss** দিয়ে ট্রেন করা হয়—সবচেয়ে কমন হলো **cross-entropy loss**।

ট্রেনিংয়ের টার্গেট:

- অডিও ইনপুট দেওয়া হলে
- সঠিক টেক্সট টোকেনগুলো প্রেডিক্ট করা

এটা অনেকটা translation/summarization ট্রেনিংয়ের মতোই—শুধু এখানে ইনপুট টেক্সট না, অডিও।

২) Inference করার সবচেয়ে সহজ উপায়: `pipeline()`

Fine-tuned (বা pretrained) ASR মডেল চালানোর সবচেয়ে সহজ উপায় হলো Transformers এর `pipeline()` ব্যবহার করা।

নিচের কোডটা দেখুন:

```
from transformers import pipeline

transcriber = pipeline(
    task="automatic-speech-recognition", model="openai/whisper-base.en"
)
transcriber("https://huggingface.co/datasets/Narsil/asr_dummy/resolve/main/mlk.flac")
# Output: {'text': ' I have a dream that one day this nation will rise up and live out the true
```

এখানে কী হচ্ছে?

- `task="automatic-speech-recognition"` বলায় pipeline বুঝে যাচ্ছে এটা ASR কাজ
- `model="openai/whisper-base.en"` দিয়ে নির্দিষ্ট Whisper checkpoint লোড হচ্ছে
- আপনি অডিও ফাইলের URL দিলে pipeline সেটা ডাউনলোড/প্রসেস করে transcript বের করে দিচ্ছে

৩) প্র্যাকটিস করতে চাইলে

আপনি যদি হাতে-কলমে ASR শিখতে চান, তাহলে পরের ধাপ হবে:

- Whisper কিভাবে fine-tune করা হয়
- কিভাবে inference আরও ভালো করা যায়

(এজন্য সাধারণত “automatic speech recognition guide” ফলো করা হয়।)

Computer Vision (কম্পিউটার ভিশন)

এখন আমরা **computer vision** টাস্কে আসি—এগুলো মূলত ছবি/ভিডিও থেকে ভিজ্যুয়াল তথ্য বোঝা এবং ব্যাখ্যা করার কাজ।

১) Computer vision টাস্ক করার দুইটা প্রধান পদ্ধতি

পদ্ধতি **A: Transformer** দিয়ে ছবি প্রসেস করা (**ViT** স্টাইল)

- ছবিকে ছোট ছোট patch এ ভাগ করা হয়
- patch গুলোকে sequence হিসেবে ধরে Transformer দিয়ে parallel প্রসেস করা হয়

পদ্ধতি **B: আধুনিক CNN** ব্যবহার করা (**ConvNeXT** স্টাইল)

- এখানে মূল অপারেশন হলো convolution
- তবে নেটওয়ার্ক ডিজাইন আধুনিক (modern design choices)

আরেকটা তৃতীয় ধরন আছে:

- Transformer + Convolution মিক্স (যেমন Convolutional ViT, LeViT)
কিন্তু এখানে আমরা সেটা ডিটেইলে যাচ্ছি না, কারণ এটা মূলত উপরের দুইটা পদ্ধতির কম্বিনেশন।

২) কোন টাস্কে কোন মডেল বেশি উপযুক্ত?

- **Image classification:** ViT এবং ConvNeXT খুব কমন
- কিন্তু অন্য টাস্ক যেমন:
 - object detection → **DETR**
 - segmentation → **Mask2Former**
 - depth estimation → **GLPN**এই টাস্কগুলোর জন্য আলাদা মডেল আর্কিটেকচার বেশি উপযোগী

Image Classification (ছবি শ্রেণিবিভাগ)

Image classification হলো computer vision-এর সবচেয়ে বেসিক টাস্কগুলোর একটি। লক্ষ্য:

- ছবিতে কী আছে সেটা নির্দিষ্ট ক্লাসে classify করা
যেমন: cat, dog, car, airplane ইত্যাদি

এখানে **ViT** এবং **ConvNeXT**—দুটোই ব্যবহার করা যায়। মূল পার্থক্য:

- **ViT:** attention mechanism ব্যবহার করে (pure Transformer approach)
- **ConvNeXT:** convolution ব্যবহার করে (modern CNN approach)

এই অংশে আমরা ViT কীভাবে কাজ করে সেটা বুঝব।

১) ViT (Vision Transformer) কীভাবে ছবিকে Transformer-এ দেয়?

ViT-এর সবচেয়ে বড় পরিবর্তন হলো:

ছবিকে টেক্সটের মতো “tokenize” করা

ধাপ ১: ছবিকে **patch** এ ভাগ করা

- একটি ছবি ভাগ করা হয় সমান আকারের, non-overlapping square patch এ
- প্রতিটি patch পরে “token” হিসেবে কাজ করে

উদাহরণ:

- 224×224 পিক্সেলের ছবি
- যদি patch size হয় 16×16
তাহলে patch সংখ্যা হবে $14 \times 14 = 196$ patch

এটা টেক্সটে যেমন sentence $\rightarrow$ token sequence হয়,
তেমনি:

image $\rightarrow$ patch sequence

ধাপ ২: **Patch embedding** তৈরি করা

প্রতিটি patch কে ভেক্টরে রূপান্তর করা হয় (patch embedding)।

এখানে উল্লেখ করা হয়েছে:

- patch embedding সাধারণত একটি convolutional 2D layer দিয়ে তৈরি করা হয়
- base Transformer-এর মতো সেটাপে প্রতিটি patch embedding dimension হয় **768** (যেমন ViT-base)

ধাপ ৩: **[CLS]** টোকেন যোগ করা (**BERT**-এর মতো)

ViT patch sequence-এর শুরুতে একটি বিশেষ learnable embedding যোগ করে:

- বিশেষ **[CLS] token**

এই [CLS] token-এর কাজ:

- পুরো ছবির একটি সারাংশধর্মী representation শেখা

শেষে:

[CLS] token-এর final hidden state-ই classification head-এ দেওয়া হয়
আর অন্য patch output সাধারণত ignore করা হয় (classification-এর ক্ষেত্রে)

ধাপ ৪: **Position embeddings** যোগ করা

Transformer নিজে থেকে বুঝতে পারে না:

- কোন patch ছবির কোন জায়গায় ছিল

তাই patch embeddings + [CLS] এর সাথে যোগ করা হয়:

- learnable **position embeddings**

এগুলো patch embedding-এর মতোই একই ডাইমেনশনের হয় এবং শেখানো যায় (learnable)।

ধাপ ৫: **Transformer encoder** দিয়ে প্রসেস

সব embedding (patch + [CLS] + position) Transformer **encoder** এ যায়।

২) Output কীভাবে **class label** হয়?

Transformer encoder থেকে আউটপুট পাওয়ার পর:

- শুধু [CLS] token-এর output নেয়া হয়
- সেটাকে পাঠানো হয় একটি **MLP head** (multilayer perceptron head) এ

MLP head কী করে?

- output → class logits এ কনভার্ট করে
- cross-entropy loss হিসাব করে
- সবচেয়ে সম্ভাব্য class নির্বাচন করে

ViT-এর pretraining objective এখানে সহজভাবে বলা হয়েছে:

- classification

৩) প্র্যাকটিস করতে চাইলে

ViT দিয়ে image classification ভালোভাবে শিখতে চাইলে সাধারণত পরের ধাপ:

- ViT fine-tune করা
- inference pipeline সেট করা

(এজন্য “image classification guide” ফলো করা হয়।)